

Documentación de Pruebas del Sistema SBAC

Sección: 2. Pruebas del Sistema

Fecha: Mayo 2026

Responsable de QA: Alan Oliver López Robles

2.1 Pruebas Unitarias para Cada Componente

Las pruebas unitarias validan el funcionamiento individual de los componentes del sistema. En este proyecto, se realizaron pruebas utilizando el framework `unittest` nativo de Python para funciones clave de la clase SBAC:

- `init()`: Crea la estructura del repositorio y la base de datos.
- `add(filepath)`: Agrega archivos al *staging area* y comprime los binarios.
- `commit(message)`: Crea un nuevo commit con los archivos agregados y genera la relación de paternidad.

Listing 1: Prueba de `init()`

Python

```
def test_01_init_creates_structure(self):
    """Prueba Unitaria: Verifica que init cree la base de datos y carpetas"""
    res = self.app.init()
    self.assertTrue(os.path.exists(sbac.core.DB_FILE))
    self.assertTrue(os.path.exists(sbac.core.OBJECTS_DIR))
    self.assertIn("Staging y Objects", res)
```

2.2 Pruebas de Integración

Estas pruebas aseguran que varios componentes funcionen juntos de manera correcta. Se validó el flujo completo desde la inicialización del repositorio hasta la comparación de código en el commit:

1. Inicialización con `init()`.
2. Agregado de archivo con `add()`.
3. Creación de commit con `commit()`.
4. Detección de cambios de código mediante `diff()`.

Listing 2: Prueba de integración

Python

```
def test_03_integration_add_commit_diff(self):
    """Prueba de Integración: Flujo completo de Add -> Commit -> Modificar -> Diff"""
    self.app.init()
    self.app.add(self.test_file)
```

```

self.app.commit("commit_inicial")

hist = self.app.history()
self.assertEqual(len(hist), 1)

with open(self.test_file, "w") as f:
    f.write("def suma(a, b, c=0):\n    return a + b + c\n")

self.app.add(self.test_file)
self.app.commit("segundo_commit")

id_v1 = hist[0]["id"]
id_v2 = self.app.history()[0]["id"]
diffs = self.app.diff(id_v1, id_v2)

self.assertTrue(len(diffs) > 0, "No se detectaron diferencias entre los commits")

```

2.3 Pruebas de Regresión Básicas

Se emplearon las pruebas existentes como base para las pruebas de regresión. Cada vez que se realiza un cambio o se añade una funcionalidad, se ejecutan todas las pruebas de manera aislada en un entorno virtual (`.sbac_test`) para garantizar la resiliencia del código:

Listing 3: Ejecución de pruebas

```

Bash
$ python -m unittest discover tests/ -v

```

Conclusión de Regresión: Esto permite validar que funcionalidades previas como `init()`, `add()`, `commit()` y los flujos de restauración (`checkout()`) continúen funcionando correctamente tras cada modificación sin arrastrar código dañado.